

Algorithmen und Datenstrukturen I SS 2009

Vorlesungsmitschrift

Prof. Dr. Ulla Kirch Prinz

Von
Michael Melchers
Alexander Untch

Inhaltsverzeichnis

1. Algorithmen und Datenstrukturen	3
1.1 Worum geht es?	3
1.2 Laufzeit von Algorithmen	6
1.3 Abstrakte Datentypen	9
2. Lineare Datenstrukturen	11
2.1 Der ADT LISTE	11
2.2 Implementierung	13
2.2.a statisch	13
2.2.b dynamisch	15
2.2.c Implementierung als Heap (Haufen)	18
2.2.d Vergleich der verschiedenen Implementierungen	21
2.3 Sortiervverfahren	22
2.3.1 „Bubble-Sort“	22
2.3.2 „Insertion-Sort“	23
2.3.3 „Selection-Sort“	24
2.4 Sortierte Listen	25
2.6 Eingeschränkte lineare Datenstrukturen	26
2.6.a Stacks (LIFO)	26
2.6.b Queues (FIFO)	28
3. Rekursion	30
3.1 Rekursive Algorithmen	30
3.2 Laufzeit rekursiver Algorithmen	30
3.3 Rekursives Sortieren von Listen	31
3.3.1 Sortieren durch Mischen	31
3.3.2 Sortieren durch Zerlegen	31
4. Bäume	33
4.1 Der ADT Tree	33
4.2 k-näre und geordnete Bäume	34
4.3 Binäre Bäume	35
4.3.1 Implementierung	36
4.3.2 Durchlaufstrategien	38
4.3.3 Binäre Suchbäume	39
4.3.4 Warteschlangen und Prioritäten	43
4.4 B – Bäume und B'-Bäume	46

1. Algorithmen und Datenstrukturen

1.1 Worum geht es?

1.2 Laufzeit von Algorithmen

1.3 Abstrakte Datentypen

Literatur:

- Nikolaus Wirth, Algorithmen u. Datenstrukturen, Teubner Verlag
- Robert Sedgewick, Algorithmen, Addison Wesley
- T. Cormen, Introduction to Algorithm, MIT Press

1.1 Worum geht es?

“Programm = Datenstruktur + Algorithmus” (N. Wirth)

Bei der Wahl von Datenstrukturen u. Algorithmen gibt es viele Möglichkeiten – allerdings bessere und schlechtere.

- a) Ein Algorithmus besteht aus endlich vielen, eindeutigen Anweisungen, die ein Rechner in einer begrenzten Zeit ausführen kann.

Beispiel: „Addiere die ganzen Zahlen von 1 bis 100.“

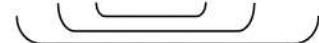
#1 Lösung:

```
unsigned long busy(int n) {
    unsigned long sum = 0;
    for (int i = 1; i <= n; i++) {
        sum += i;
    }
    return sum;
}
```

Alternative:

K. F. Gauss (geb. 1777)

$$1 + 2 + 3 + \dots + 98 + 99 + 100 = (n + 1) * n/2 \quad n = 100$$



#2 Lösung:

```
unsigned long gauss(int n) {
    return (unsigned long)(n+1) * n/2;
}
```

Typecast um Überlauf zu vermeiden!

Vergleich:

#1 Lösung: braucht n Additionen + Overhead der for-Schleife

#2 Lösung: braucht 3 arithmetische Operationen + Typecast

Wichtig:

Ein Algorithmus ist besser als ein anderer wenn er in kürzerer Zeit ausgeführt werden kann.

b) Eine Datenstruktur legt fest, wie die Daten in einem Programm dargestellt werden.

Beispiel:

„Bilde Datenstruktur zur Darstellung von Skatkarten für die Simulation eines Skatspieles.“

#1 Lösung:

```
enum Farbtyp {Kreuz, Pik, Karo, Herz};
enum Werttyp {Sieben, ... , As};

struct Karte {
    Farbtyp farbe;
    Werttyp wert;
};

Karte blatt[10];
```

Speicherbedarf für ein Blatt:

$$10 * (4 + 4) = 80 \text{ Byte}$$

Alternative:

Codierung der Karten

	7	8	9	...	König	As
Kreuz	1	2	3			8
Pik	9					16
Karo	17					24
Herz	25					32

#2 Lösung:

```
unsigned short blatt[10];
```

Speicherbedarf für ein Blatt:

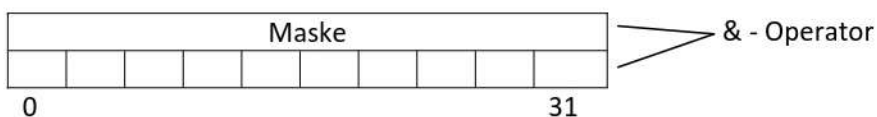
$$10 * 2 = 20 \text{ Byte}$$

Alternative:

Codierung der Karten (wie zuvor).

#3 Lösung:

```
unsigned int blatt; // 4 Byte = 32 Bit
```



Regel: Die Karte mit der Nr. i ist im Blatt enthalten
 $\Rightarrow$ das $(i - 1)$ Bit in der Variablen **blatt** ist gesetzt.

Speicherbedarf für ein Blatt:

4 Byte

Frage: Wann ist eine Datenstruktur besser als eine andere?

Antwort: Man hat zwei Kriterien:

1. Speicherbedarf
2. Geschwindigkeit der Operationen

Beispiel:

„Suche eine bestimmte Karte in einem Blatt“

- #1: Beginnend bei der ersten Karte:
Farbe und Wert vergleichen $\Rightarrow$ im schlimmsten Fall 20 Einzelschritte
- #2: Sequentielle Suche $\Rightarrow$ max. 10 Suchschritte
- #3: i-tes Bit in der Variablen **blatt** gesetzt?

Zusammenfassung:

Wahl geeigneter Datenstruktur entscheidend für:

- rationellen Speicherverbrauch
- Geschwindigkeit entsprechender Algorithmen

$\Rightarrow$ Algorithmen + Datenstruktur wesentlich für effiziente Programme

1.2 Laufzeit von Algorithmen

Die Laufzeit eines Programms hängt ab

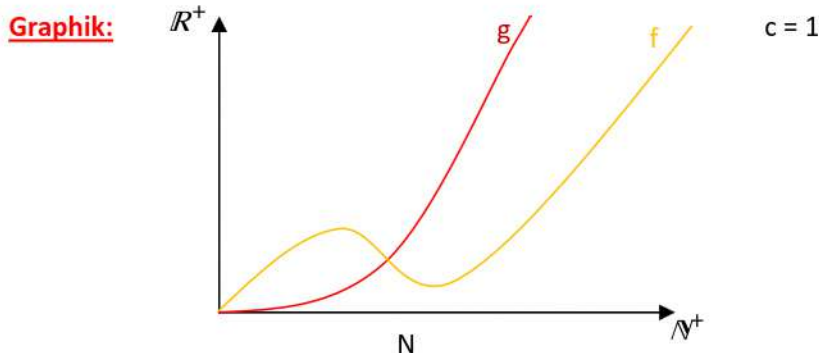
1. Von der verwendeten Hardware und vom jeweiligen Compiler / Interpreter.
2. Von der Anzahl der Eingabedaten
3. Der Art der Eingabedaten (int < double)

Zur Abschätzung der Laufzeit eines Programms sind Funktionen nützlich die

- Von der Hardware, vom jeweiligen Compiler und der Art der Eingabedaten unabhängig sind, d. h. konstante Faktoren bleiben unberücksichtigt.
- Von der Anzahl der Eingabedaten abhängig.
- Eine obere Grenze für die Laufzeit angeben, d. h. den schlimmsten Fall berücksichtigen.

a) Das Größenordnungsmaß O („Big-O“)

Definition: Sind $f : \mathbb{N} \rightarrow \mathbb{R}^+$ und $g : \mathbb{N} \rightarrow \mathbb{R}^+$ zwei Funktionen, dann sei
 $f \in O(g) \Leftrightarrow \exists c > 0, \exists N \in \mathbb{N}$
 $f(n) \leq c * g(n) \quad \forall n > N$



Regeln:

1. $k * f \in O(f)$
 setze $c := k, N := 1$
2. $f \in O(g), g \in O(h) \Rightarrow f \in O(h)$
 setze $c := c_1 * c_2, N := \max(N_1, N_2)$

$\downarrow$
 f

$\downarrow$
 g

$\uparrow$
 f

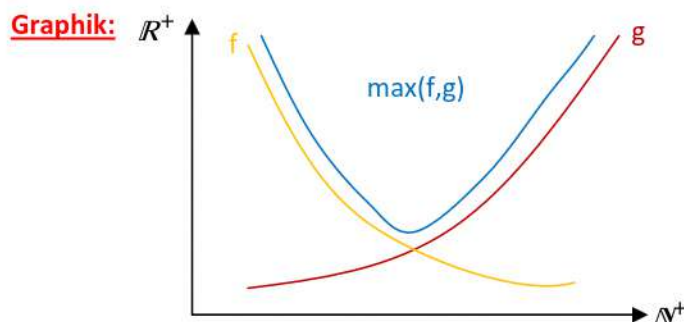
$\uparrow$
 g

Dann gilt:

$$f(n) \leq c_1 * g(n) \leq c_1 * c_2 * h(n) = c * h(n) \quad \forall n > N$$

$$\Rightarrow f \in O(h)$$

3. $f + g \in O(\max(f, g))$
 wobei $\max(f, g)(n) := \max(f(n), g(n))$



$$4. f_1 \in O(g_1), f_2 \in O(g_2) \Rightarrow f_1 * f_2 \in O(g_1 * g_2)$$

Bemerkung:

Mit diesen Regeln können Abschätzungen für die Laufzeit vereinfacht werden.

Merke:

1. Die Exponentialfunktion wächst stärker als jedes Polynom
 $\Rightarrow O(n^{1000})$ – Algorithmen sind schneller als $O(2^n)$ – Algorithmen.
2. Die Logarithmus-Funktion wächst langsamer als jedes Polynom
 $\Rightarrow O(\log_2(n))$ – Algorithmus ist schneller als $O(n)$ – Algorithmen.

b) Big-O Abschätzungen von Algorithmen

Voraussetzungen:

1. Jede Anweisung benötigt dieselbe Zeit **Modulo** einer Konstanten.
2. Jeder if-Anweisung braucht so viel Zeit wie ihr längster Zweig.
3. Die Anzahl der Schleifendurchläufe ist immer maximal

1. Beispiel: „Einfache Schleife“

```

int search(long key, long v[], int n) {
    int wo = -1;
    for (int i = 0; i < n; i++) {
        if (v[i] == key)
            wo = i;
    }
    return wo;    // -1 wenn key nicht in v[]
}

```

Complexity analysis for the first example:

- $O(1)$ for the initialization `int wo = -1;`
- $O(n)$ for the `for` loop and its body (indicated by a bracket on the left).
- $O(1)$ for the `return` statement.

2. Beispiel: „Geschachtelte Schleife“

```

long prodsum(int n) {
    long result = 0;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            result += (long)i*j;
        }
    }
    return result;
}

```

Big-O Abschätzung:

Wert von i:	1	2	3	...	n
Anzahl innere Schleifendurchläufe:	1	2	3	...	n
Gesamtaufwand:	1+	2+	3+	...+	$n = \frac{n*(n-1)}{2} = \frac{n^2}{2} + \frac{n}{2} \in O(n^2)$

3. Beispiel: „Geschachtelte Schleife“

```
int m = 1;
for (int i = 1; i <= n; i++) {
    m *= 2;
    for (int j = 1; j <= m; j++) {
        // tu was element O(1)
    }
}
```

Big-O Abschätzung:

Wert von i:	1	2	3	...	n
Anzahl innere Schleifendurchläufe:	2	2^2	2^3		2^n
Gesamtaufwand	2+	$2^2 +$	$2^3 +$	...	$2^n = 2 * (2^0 + 2^1 + \dots + 2^{n-1}) =$ $= 2 * (\frac{2^n - 1}{2 - 1}) \in O(2^n)$

1.3 Abstrakte Datentypen

Ein abstrakter Datentyp (ADT) ist die Abstraktion von Daten und Operationen in einem Programm.

Man hat drei Ebenen:

Informelle Darstellung von Daten

- ADT -

Implementierung in einem Programm



a) Relationen

Definition:

Eine Relation auf einer Menge P ist eine Funktion $R : P \times P \rightarrow \text{Boolean}$

Bezeichnung:

Falls $R(x, y) = \text{true}$, so schreibt man xRy

Beispiel:

1. $P = \mathbb{N}$, $R(x, y) = \text{true}$ falls $x < y$
2. $P = \mathbb{Z}$, $R(x, y) = \text{true}$ falls x teilt y ohne Rest

Bezeichnung:

Eine Relation R auf P heißt

1. **reflexiv**, falls $xRx \ \forall x \in P$ gilt
2. **symmetrisch**, falls $\forall x, y \in P$ gilt: $xRy \Rightarrow yRx$
3. **antisymmetrisch**, falls aus xRy und yRx folgt: $x = y \ \forall x, y \in P$
4. **transitiv**, falls aus xRy und yRz folgt $xRz \ \forall x, y, z \in P$
5. eine **Kette**, falls $x, y \in P$ xRy oder yRx gilt

Beispiel:

1. „=“ reflexiv, symmetrisch, transitiv
2. „/“ (Teiler-Relation) reflexiv, antisymmetrisch, transitiv

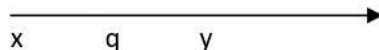
Definition:

Es seien $x \in P$, $q \in P$, dann heißt q :

- **unmittelbarer Nachfolger** von x , wenn gilt:

1. xRq („ q kommt nach x “)
2. Ist $y \in P$ mit $y \neq x$, $y \neq q$ und $xRy \Rightarrow qRy$

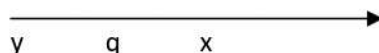
Graphik:



- **unmittelbarer Vorgänger** von x , wenn gilt:

1. qRx („ q kommt vor x “)
2. Ist $y \in P$ mit $y \neq x$, $y \neq q$ und $yRx \Rightarrow yRq$

Graphik:



Definition: Eine Relation R auf P heißt **lineare Ordnung**, falls R reflexiv, antisymmetrisch, transitiv und eine Kette ist.

Beispiel: $\leq, \geq$

Satz:

Ist eine **endliche Menge** und R eine lineare Ordnung auf P , so gilt:

- o Es gibt genau ein Element in P , das keinen Vorgänger hat: das „**erste** Element“.
Alle anderen Elemente haben genau einen unmittelbaren Vorgänger.
- o Es gibt genau ein Element in P das keinen Nachfolger hat: das „**letzte** Element“.
Alle anderen Elemente haben genau einen unmittelbaren Nachfolger.

Graphik: o oooooooooooooooooooooo o

b) Definition eines ADT

Sämtliche Datenstrukturen können aus einem einzigen Klassifikationsschema hergeleitet werden. Dieses Schema beschreibt die Objekte eines ADT's, ihre Struktur und elementare Operationen.

Definition:

Ein ADT besteht aus

- o Einer Menge **ATOM** (Informationen) und einer Menge **POSITION** (z.B. Indexmenge)
- o Einer Menge von Tripeln (P, R, v) des ADT's, für die gilt:
 - P ist eine endliche Teilmenge von **POSITION**
 - R ist eine Relation auf P
 - $V: P \rightarrow \text{ATOM}$ ist eine Funktion, die sog. Auswertungsfunktion
- o Einer Menge von elementaren Operationen

Beispiel: Der ADT String

- o **ATOM** Menge der darstellbaren Zeichen, z.B. Unicode
- o **POSITION** : $\mathbb{N}$
- o Tripel (P, R, v) mit
 $P = \{1, 2, \dots, n\}$
Relation $R : \leq$
 $v(i)$ liefert Zeichen in Position i
- o Operationen (elementar): Create, Append, DeleteFirst, Retrieve

Bemerkung: Die Operationen sind in dem Sinne elementar, als durch sie alle anderen Operationen zusammengesetzt werden können.

2. Lineare Datenstrukturen

- 2.1 Der ADT LISTE
- 2.2 Implementierungen
- 2.3 Sortiervverfahren für lineare Listen
- 2.4 Sortierte Listen
- 2.5 Anwendungsbeispiele
- 2.6 Eingeschränkte lineare Datenstrukturen (Stacks, Queues)

2.1 Der ADT LISTE

Definition:

Der ADT LISTE besteht aus:

- Mengen **ATOM** und **POSITION**
- Elemente (P, R, v) des ADT's
 - o P, v (wie gehabt)
 - o R ist eine lineare Ordnung
- Elementare Operationen vgl. Kopie

Elementare Operationen für den ADT LISTE (Kopie):

Create	Erzeugt eine leere Liste, bzw. Kürzt eine bereits existierende Liste auf Länge 0.
Empty	Prüft, ob eine Liste leer ist.
First	Bestimmt die erste Position in einer Liste.
Last	Bestimmt die letzte Position in einer Liste.
Next	Bestimmt zu einer vorgegebenen Position in einer Liste den unmittelbaren Nachfolger.
Prior	Bestimmt zu einer vorgegebenen Position in einer Liste den unmittelbaren Vorgänger.
OK	Prüft, ob eine vorgegebene Position in einer Liste ist.
Retrieve	Liefert das Atom in einer vorgegebenen Position.
Update	Ersetzt das Atom in einer vorgegebenen Position durch ein anderes Atom.
DeletePos	Entfernt das Atom in einer vorgegebenen Position, wobei die Listenstruktur erhalten bleibt.
InsertAfter	Fügt ein Atom unmittelbar hinter eine vorgegebene Position in eine Liste ein.

Beispiele für nicht elementare Operationen (Kopie):

Find	Bestimmt die erste Position, wo ein vorgegebenes Atom in der Liste auftritt.
Insert	Fügt ein Atom in eine Liste ein.
Delete	Löscht ein Atom in einer Liste.
Sort	Sortiert die Listenelemente nach einem vorgegebenen Schlüssel.

Vektorielle Implementierung einer linearen Liste (Kopie):

```
// Header-Datei vekliste.h:
// Enthält die Klasse VekList zur Darstellung einer linearen Liste, die
// vektoriell implementiert wird.

#define PSEUDO -1          // Index, der nicht vorkommt

typedef int POSITION;      // Indexmenge.
typedef int ATOM;         // Datentyp fuer Infos.

class VekList
{
private:
    ATOM *vekPtr;         // Zeiger auf Vektor.
    int anz;              // Aktuelle Anzahl Vektorelemente.
    int max;              // Maximale Anzahl Vektorelemente.

public:
    VekList(int n);                // Kontruktor
    VekList(const VekList& ref);    // Kopierkonstruktor
    VekList& operator=(const VekList& ref); // Zuweisungsoperator
    ~VekList();                    // Destruktor

    // Hier keine Zugriffsmethoden...

    // Elementare Methoden
    bool Empty(void);
    POSITION First(void);
    POSITION Next(void);
    POSITION Last(void);
    POSITION Prior(POSITION i);
    bool ok(POSITION i);

    ATOM Retrieve(POSITION i);
    void InsertAfter(POSITION newAtom, POSITION i);
    void DeletePos(POSITION i);
    void Update(ATOM upAtom, POSITION i);
};
```

2.2 Implementierung

2.2.a statisch

Es wird ein zusammenhängender Speicherbereich fester Länge angelegt, in dem die Listenelemente unmittelbar hintereinander abgelegt werden: Vektor

Klassendefinition: vgl. Kopie (01.04.2009)

```
int x = 7;
```

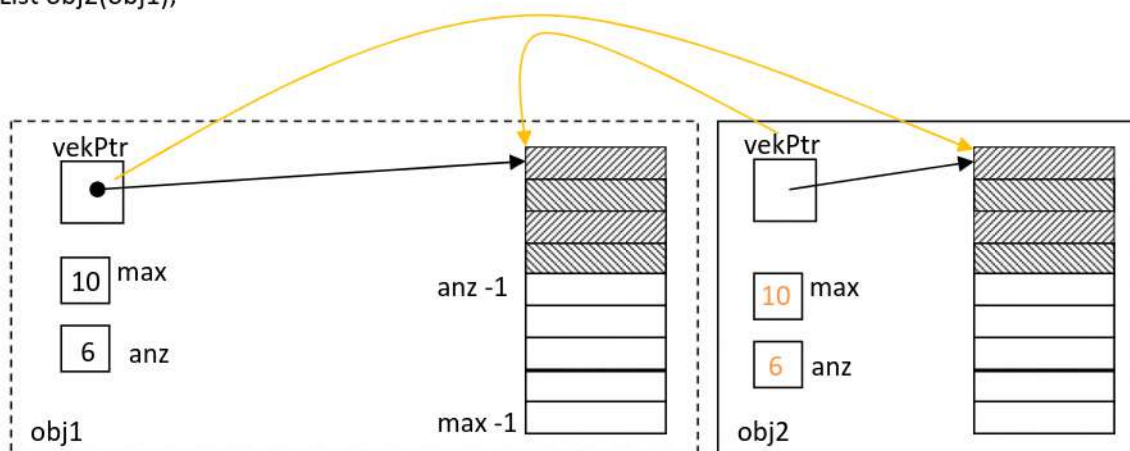
```
int y = x;
```



```
VekList obj1(...);
```

```
VekList obj2(obj1);
```

```
obj1 = obj2;
```



Implementierung von Methoden:

```
// Konstruktor
VekList::VekList(int n)
{
    vekPtr = new ATOM[n];
    anz = 0;
    max = n;
}

// Destruktor
VekList::~~VekList()
{
    delete[] vekPtr;
}

// Kopierkonstruktor
VekList::VekList(const VekList& ref)
{
    anz = ref.anz;
    max = ref.max;
    vekPtr = new ATOM[max];
    for(int i = 0; i < anz; i++)
        vekPtr[i] = ref.vekPtr[i];
}
```

```

// Zuweisungsoperator
VekList& VekList::operator=(const VekList& ref)
{
    delete[] vekPtr;
    anz = ref.anz;
    max = ref.max;
    vekPtr = new ATOM[max];
    for(int i = 0; i < anz; i++)
        vekPtr[i] = ref.vekPtr[i];
    return *this;
}

// Einfügen
void VekList::InsertAfter(ATOM newAtom, POSITION i)
{
    if(anz < max)
    {
        for(int j = anz; j > i+1; j--)
            vekPtr[j] = vekPtr[j-1];

        vekPtr[i+1] = newAtom;
        anz += 1;
    }
    else
    {
        // Andernfalls kein Platz.
        cout << "Kein Platz mehr!" << endl;
    }
} // Laufzeit: O(n)

// Löschen
void VekList::DeletePos(POSITION i)
{
    // if(anz < max)
    // {
        for(int j = i; j < anz-1; j++)
            vekPtr[j] = vekPtr[j+1];

        anz -= 1;
    // }
} // Laufzeit: O(n)

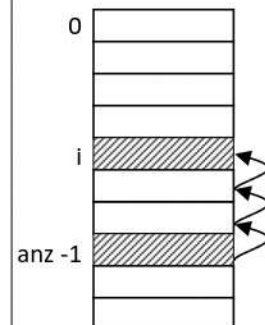
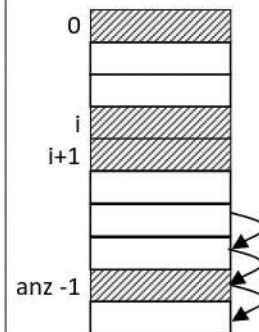
// Main
#include <iostream>
#include "VekList.h"

using namespace std;

int main()
{
    VekList a(10);
    // Nicht: VekList* ap = new VekList(10);

    a.InsertAfter(100, PSEUDO);
}

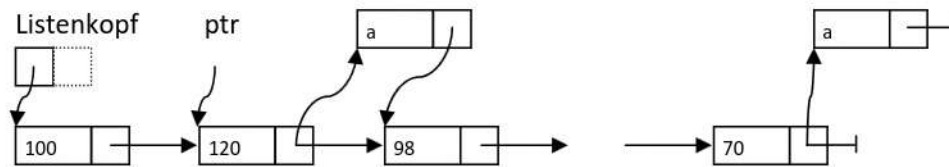
```



2.2.b dynamisch

Eine lineare Liste wird als Folge von verketteten Zellen dargestellt, wobei jede Zelle ein Atom und einen Zeiger auf den unmittelbaren Nachfolger enthält.

Graphik:



Klassendefinition: vgl. Kopie

Dynamische Implementierung einer linearen Liste (1) (Kopie):

```
// Header-Datei listel.h
// Enthält die Klassen Cell und ListEl
// zur Darstellung der Elemente einer linearen Liste, die dynamisch
// implementiert wird.

#include <cstdio>           // Def. der symb. Konstanten NULL.
using namespace std;

#define PSEUDO NULL        // Pseudo-Position: NULL-Zeiger.

typedef int ATOM;          // Datentyp fuer Infos.

// Die Klasse Cell stellt das "Geruest" zur Verwaltung eines
// Listenelements dar.
class Cell {
private:
    Cell *suc;

public:
    // inline definierte Methoden:
    Cell(Cell *p) { suc = p; }
    virtual ~Cell() { }
    void SetSuc(Cell *p) { suc = p; }
    Cell* GetSuc(void) { return suc; }
};

// Die Klasse ListEl stellt ein Listenelement dar.
class ListEl : public Cell {
private:
    ATOM atom;

public:
    // Zwei ueberladene Konstruktoren:
    ListEl(ATOM a) : Cell(PSEUDO) { atom = a; }
    ListEl(ATOM a, Cell *ptr) : Cell(ptr) { atom = a; }

    // Zugriffsmethoden inline
    ATOM GetAtom(void) { return atom; }
    void SetAtom(ATOM neu) { atom = neu; }
};

typedef Cell* POSITION;      // Typ fuer Basisklassenzeiger
```

Dynamische Implementierung einer linearen Liste (2) (Kopie)

```
// Header-Datei ptrliste.h
// Enthält die Klasse PtrList zur Darstellung einer linearen Liste, die
// dynamisch implementiert wird.

#include "listel.h"           // Def. der Klassen Cell, ListEl.

class PtrList {
private:
    POSITION start;           // Zeiger auf erstes Listenelement.
    int anz;                 // aktuelle Anzahl Listenelemente.

    // Hilfsmethoden:
    void eraseAll();
    void copyAll(const PtrList& ref);

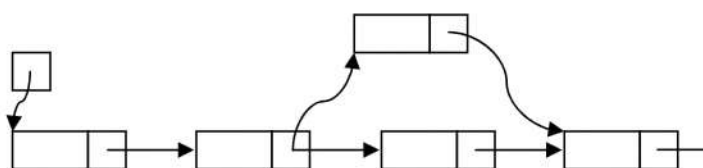
public:
    PtrList() { start = PSEUDO; anz = 0; }           // Konstruktor
    PtrList(const PtrList& ref) { copyAll(ref); }     // Kopierkonstruktor
    ~PtrList() { eraseAll(); }                       // Destruktor
    PtrList& operator=(const PtrList& ref) {          // Zuweisung
        eraseAll();
        copyAll(ref);
        return *this;
    }

    // Elementare Hilfsmethoden:
    bool Empty(void);
    POSITION First(void);
    POSITION Next(POSITION ptr);
    POSITION Last(void);
    POSITION Prior(POSITION ptr);
    bool ok(POSITION ptr);

    ATOM Retrieve(POSITION ptr);
    void InsertAfter(ATOM neu, POSITION ptr);
    void DeletePos(POSITION ptr);
    void Update(ATOM newKey, POSITION ptr);
};
```

Implementierung:

```
// Einfügen
void PtrList::InsertAfter(ATOM a, POSITION ptr) {
    POSITION p;
    if (ptr == PSEUDO) {           // vor allen anderen einfügen
        p = new ListEl(a, start);
        start = p;
    } else {                       // in der Mitte oder am Ende einfügen
        p = new ListEl(a, Next(ptr));
        ptr->SetSuc(p);           // (*ptr).SetSuc(p);
    }
    anz += 1;
    // Laufzeit: O(1)
}
```

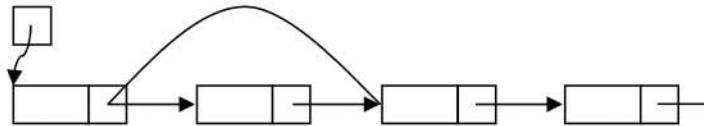


Einfügen Laufzeit: $O(1)$

```

void PtrList::DeletePos(POSITION p) {
    if (anz > 0) {
        POSITION prev = Prior(p);    // Vorgänger
        if (prev == PSEUDO)          // 1. Element löschen
            start = Next(start);
        else
            prev->SetSuc(Next(p));  // Zeiger umbiegen
        delete p;
        anz -= 1;
    }
}

```



```

ATOM PtrList::Retrieve(POSITION p) {
    return ((ListEl*)p)->GetAtom();
}

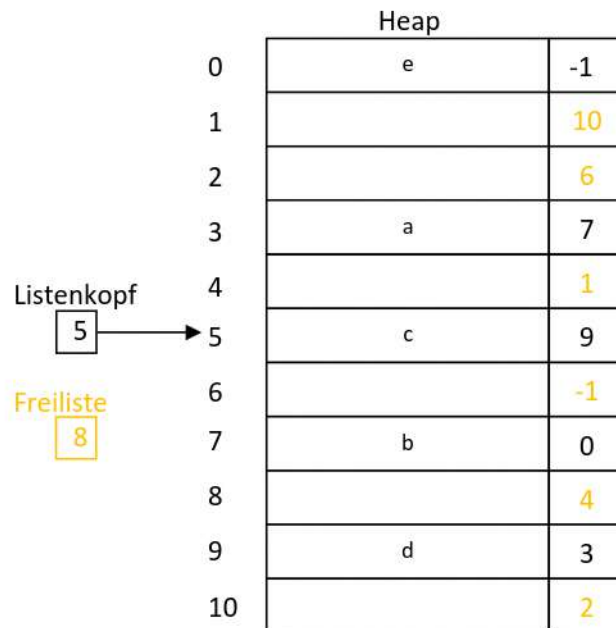
void PtrList::eraseAll()
{
    POSITION p;
    while (start != PSEUDO) {
        p = start;
        start = Next(start);
        delete p;
    }
    anz = 0;
    // Laufzeit: O(n)
}

```

2.2.c Implementierung als Heap (Haufen)

Es wird ein Vektor gebildet also ein zusammenhängender Speicher fester Länge, der sog. Heap. Jedes Vektorelement enthält ein Atom und einen Index des unmittelbaren Nachfolgers in der Liste.

Graphik:



Bei dieser Darstellung ist es sinnvoll, eine Freiliste zu unterhalten, die alle freien Plätze im Heap verwaltet.

Beim Einfügen eines Elements wird der erste freie Platz der Freiliste entnommen.

Beim Löschen eines Listenelements kann dieses als erstes Element in die Freiliste eingefügt werden.

Klassendefinition: vgl. Kopie

Cursor-Implementierung einer linearen Liste mit Freispeicherverwaltung (Kopie):

```
// Header-Datei curliste.h
// Enthält die Klassen Heap, FreeList und CurList
// zur Cursor-Implementierung einer linearen Liste mit
// Freispeicherverwaltung.

#define PSEUDO -1                // Index, der nicht vorkommt.

typedef int POSITION;             // Indexmenge.
typedef int ATOM;                // Datentyp fuer Infos.

struct HeapEl {
    ATOM data;
    POSITION suc;
};

class Heap {
private:
    HeapEl *heapPtr;             // Zeiger auf Heap.
    int max;                     // Maximale Anzahl Heap-Elemente.

public:
    Heap(int n);                 // Konstruktor.
    Heap(const Heap& ref);        // Kopier-Konstruktor.
    Heap& operator=(const Heap& ref); // Zuweisungsoperator
    ~Heap();                     // Destruktor.

    // Zugriffsmethoden:
    int GetMax(void) { return max; }

    void SetSuc(POSITION p, POSITION s);
    POSITION GetSuc(POSITION p);

    void SetAtom(POSITION p, ATOM a);
    ATOM GetAtom(POSITION p);

    void Show(void);              // Zum Anschauen.
};

class FreeList : public Heap {
private:
    POSITION top;                 // Erstes Element der Freiliste.
    int size;                    // Anzahl Elemente in der Freiliste.

public:
    FreeList(int n);
    ~FreeList();                 // Erforderlich?

    int GetSize(void) { return size; }
    bool EmptyFree() { return (bool)(size == 0); }
    void AddToFree(POSITION p);  // Element einfügen.
    POSITION GetFromFree(void);    // Freies Element holen.
};
```

```

class CurList : public FreeList {
private:
    POSITION start;           // Erstes Element in der linearen Liste.
    int len;                 // Anzahl Listenelemente.

public:
    CurList(int n);
    ~CurList();             // Erforderlich?

    // Elementare Methoden:
    bool Empty(void) { return (bool)(len == 0); }
    POSITION First(void) { return (start); }
    POSITION Next(POSITION i);
    POSITION Last(void);
    POSITION Prior(POSITION i);
    bool ok(POSITION i);

    ATOM Retrieve(POSITION i) { return GetAtom(i); }
    void InsertAfter(ATOM newAtom, POSITION i);
    void DeletePos(POSITION i);
    void Update(ATOM neu, POSITION i) {
        if (ok(i))
            SetAtom(i, neu);
    }
};

```

Implementierung der Methoden von Heap:

```

Heap::Heap(int n) {
    heapPtr = new HeapEl[n];
    max = n;
}

Heap::~~Heap() {
    delete[] heapPtr;
}

```

Implementierung der Methoden von FreeList:

```

FreeList::FreeList(int n) : Heap(n) {
    top = 0;
    size = n;
    for (int i = 0; i < size-1; i++) {
        SetSuc(i, i+1);
    }
    SetSuc(size-1, PSEUDO);
}

void FreeList::AddToFree(POSITION p) {
    SetSuc(p, top);
    top = p;
    size += 1;
} // Laufzeit: O(1)

```

```

POSITION FreeList::SetFromFree() {
    if (size > 0) {
        POSITION temp = top;
        top = GetSuc(top);
        size -= 1;
        return temp;
    } else {
        // Fehler: kein Platz!
    }
} // Laufzeit: O(1)

```

Methoden von CurList:

Wie verkettete Listen und AddToFree() bzw. GetFromFree() einsetzen!

2.2.d Vergleich der verschiedenen Implementierungen

	Statisch	Dynamisch	Heap
Einfügen	$O(n)$	$O(1)$	$O(1)$
Löschen	$O(n)$	$O(n)$	$O(n)$
Update	$O(1)$	$O(1)$	$O(1)$
Suchen	$O(n)$	$O(n)$	$O(n)$

2.3 Sortierverfahren

Sortieren durch direktes

Austauschen „Bubble-Sort“
 Einfügen „Insertion-Sort“
 Auswählen „Selection-Sort“

Später: „Quick-Sort“, „Merge-Sort“, „Heap-Sort“

2.3.1 „Bubble-Sort“

Verfahren: Bei einem Durchlauf werden benachbarte Elemente verglichen und ggf. getauscht.
 ⇒ Das jeweils größte Element wandert nach „oben“.

Beispiel:

1. Durchlauf:

100	120	30	40
-----	-----	----	----

100	30	40		120
-----	----	----	--	-----

 rechts ist der sortierte Teil des Arrays

Methode:

```
void VekList::BubbleSort() {
    POSITION temp;
    POSITION final = Last();    // final ist letzte Position
    ATOM help;

    while (final > 1) {
        for (temp = 0; temp < final; temp++) {
            if (vekPtr[temp] > vekPtr[temp+1]) {
                // Atome tauschen
                help = vekPtr[temp];
                vekPtr[temp] = vekPtr[temp+1];
                vekPtr[temp+1] = help;
            }
            final -= 1;
        }
    }
}
```

Geeignet für: statische, dynamische und Cursor – Implementierungen.

Laufzeit: $O(n^2)$

Anzahl Vergleiche: $(n-1) + (n-2) + \dots + 2 + 1 = n * \frac{n-1}{2}$
 1. Durchl. 2. Durchl.

Anzahl Bewegungen von Atomen:

„best case“:	0
„worst case“:	$n * \frac{n-1}{2}$
„average case“:	$\frac{1}{2} * n * \frac{n-1}{2}$

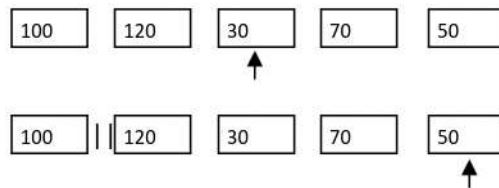
Optimierungen:

1. Flag setzen, sobald kein Tausch statt gefunden hat ⇒ fertig.
 2. Wechsel der Durchlaufrichtung nach jedem Durchlauf: „Shaker-Sort“.
- „Insertion-„ und „Selection-Sort“ besser!

2.3.3 „Selection-Sort“

Verfahren: Suche das kleinste Element in der Liste und tausche es mit dem ersten Element.
Suche das zweitkleinste Element und tausche es mit dem zweiten usw.

Beispiel:



Vorteil: Wenig Bewegungen von Atomen.

Methode:

```
void vekList::SelectionSort() {
    POSITION merke;
    POSITION last = Last();
    ATOM help;

    for (POSITION i = 0; i < last; i++) {
        merke = i;
        // Suche kleineres Element
        for (POSITION j = i+1; j <= last; j++) {
            if (vekPtr[j] < vekPtr[merke]) {
                merke = j;
            }
        }
        // Atome tauschen
        help = vekPtr[i];
        vekPtr[i] = vekPtr[merke];
        vekPtr[merke] = help;
    }
}
```

Laufzeit: $O(n^2)$

2.4 Sortierte Listen

Der ADT SORT_LIST ist eine lineare Liste, für die gilt:

1. ATOM ist eine Menge mit einer linearen Ordnung R_A
2. Die lineare Ordnung R auf P ist mit der linearen Ordnung R_A auf ATOM kompatibel, d.h.
 $pRq \Rightarrow v(p)R_A v(q)$

Bemerkung:

Beim Einfügen von Atomen muss die Liste sortiert bleiben:

Es ist also zunächst die Position in der Liste zu bestimmen, wo das Element eingefügt wird.

- i) Sequentielle Suche $O(n)$
- ii) Binäres Suchverfahren:
Das gesuchte Atom wird mit dem mittleren Element verglichen.
Ist es kleiner als das mittlere Element -> links weiter suchen.
Andernfalls: rechts weiter suchen.

Laufzeit: Das Intervall $[0, n - 1]$ kann so oft halbiert werden, wie n durch 2 teilbar ist:
 $\frac{n}{2}, \frac{n}{4}, \frac{n}{8}, \dots, \frac{n}{2^k}$ mit $2^k > n$
Stop: so bald $k \geq \log_2(n) \Rightarrow O(\log_2(n))$

„To get a feeling“:

2^{30} Atome

$\Rightarrow$ 30 Suchschritte sind erforderlich um 1 Atom unter 1,1 Milliarden Atomen zu lokalisieren.

Einfügen in sortierte Listen:**i) Statisch**

1. Binäre Suche der Einfügeposition.
2. Verschiebe alle Elemente ab der Einfügeposition, um Platz zu machen.
3. Element einfügen

Laufzeit: $O(\underbrace{\log_2(n)}_{\text{Suche}} + \underbrace{n}_{\text{Shift}} + \underbrace{1}_{\text{Einfügen}}) = O(n)$

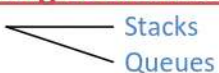
ii) Dynamisch

1. Sequentielle Suche der Einfügeposition
2. Zeiger umbiegen

Laufzeit: $O(\underbrace{n}_{\text{Suche}} + \underbrace{1}_{\text{Einfügen}}) = O(n)$

Multilisten: vgl. 1. Aufgabe

2.6 Eingeschränkte lineare Datenstrukturen

Hier:  Stacks
Queues

Vorteil: Elementare Operationen haben konstante Laufzeit.

2.6.a Stacks (LIFO)

Ein Stack ist eine lineare Datenstruktur bei der Einfügen, Löschen und Hervorholen von Atomen an **einem** Ende von der Liste vorgenommen wird.

Der ADT STACK ist eine lineare Liste mit folgenden elementaren Operationen:

- **Create:** erzeugt leeren Stack.
- **Push:** fügt ein Atom am Beginn des Stacks ein.
- **Pop:** entfernt das Atom am Beginn des Stacks.
- **Top:** holt das Atom am Beginn hervor.
- **Empty:** wie gehabt.

Anwendungen:

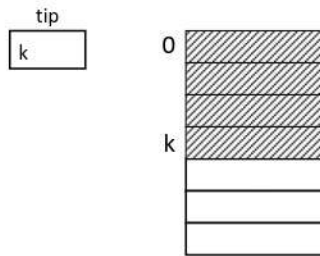
- Verwaltung von freien Speicherblöcken durch Freilisten.
- Lokale Variablen und Rücksprungadressen werden auf den Stack des Programms gepusht.

Implementierung von Stacks:

i) Statisch

Wichtig: Merke die Einfüge- / Lösch – Position.

Graphik:



Klassendefinition:

```
class VekStack {
private:
    ATOM *vekPtr;
    POSITION tip;    // Spitze / am Anfang -1!
    int max;

public:
    // Konstruktor, Destruktor usw:

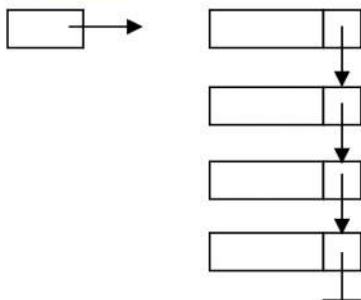
    void Push(const ATOM& a) {
        if (tip < max-1) {
            vekPtr[++tip] = a;
        }
    }

    void Pop() {
        if (tip != PSEUDO) {
            tip--;
        }
    }

    ATOM Top() {
        if (tip != PSEUDO) {
            return vekPtr[tip];
        }
    }
};
```

Nachteil: Stack kann überlaufen!

ii) Dynamisch



2.6.b Queues (FIFO)

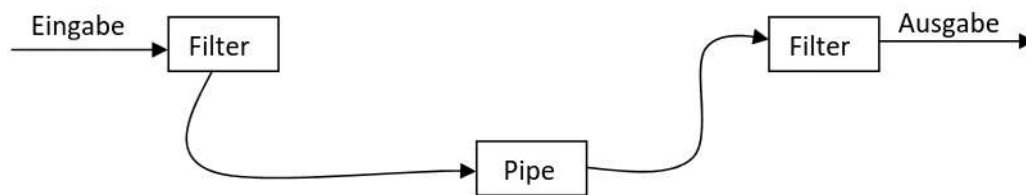
Eine Queue ist eine lineare Datenstruktur, bei der Einfügen am Ende, Löschen und Hervorheben am Anfang vorgenommen wird.

Der ADT QUEUE ist eine lineare Liste mit folgenden elementaren Operationen:

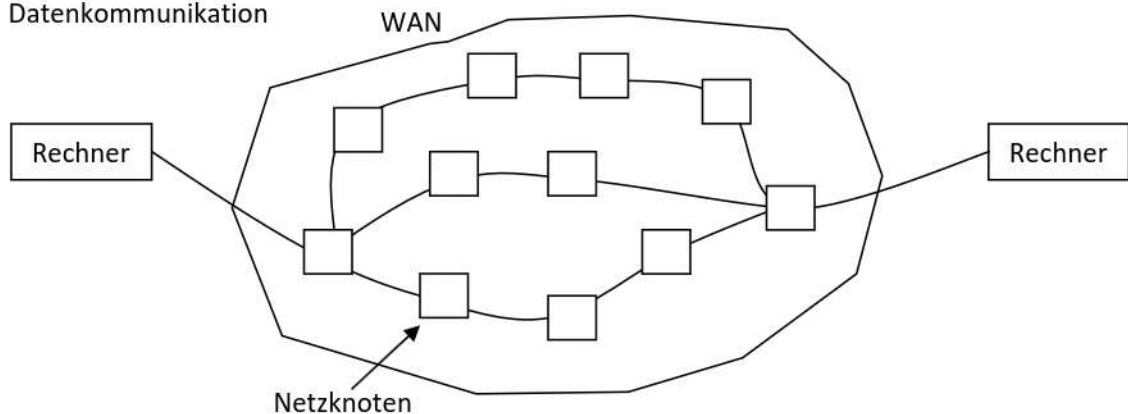
- **Create, Empty:** wie gehabt.
- **Enqueue:** fügt ein Atom am Ende ein.
- **Dequeue:** löscht das Atom am Anfang.
- **Head:** holt das Atom am Anfang hervor.

Anwendungen:

- Dateipuffer, insbes. Drucker- Tastaturpuffer.
- Pipes unter UNIX



- Datenkommunikation



In den Netzknoten werden Datenpakete in Queues zwischen gespeichert.

Implementierung von Queues:

i)

Statisch

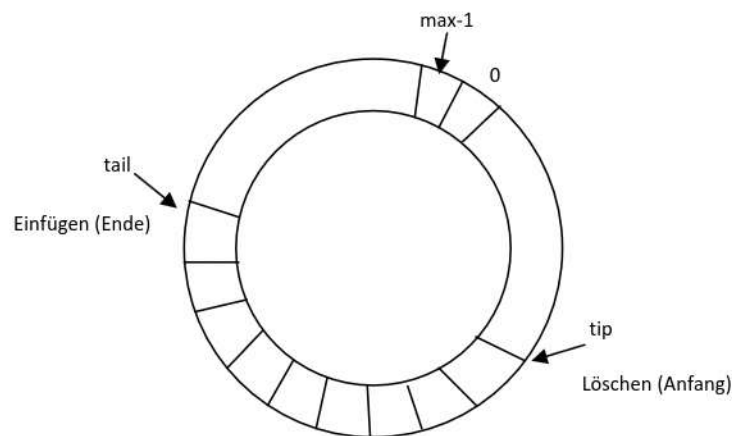


Anfang (Löschen)

Ende (Einfügen)

Queue „läuft“ Richtung Ende des Vektors.

Um einen Shift zu vermeiden, Implementierung als Ring:



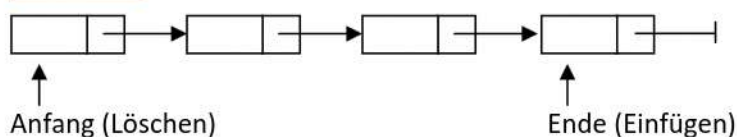
Klassendefinition:

```
class vekQueue {
private:
    ATOM *vekPtr;
    int max;
    POSITION tip, tail;
    bool empty;

public:
    void Enqueue(const ATOM& a) {
        if (empty || tip != tail) { // Platz?
            vekPtr[tail] = a;
            tail = (tail+1) % max;
            empty = false;
        }
    }

    void Dequeue() {
        if (!empty) {
            tip = (tip+1) % max;
            if (tip == tail) {
                empty = true;
            }
        }
    }
};
```

ii) Dynamisch



Merke: Adresse erstes und letztes Element.

3. Rekursion

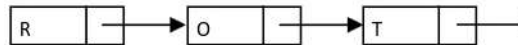
3.1 Rekursive Algorithmen

Definition: Ein Algorithmus heißt rekursiv, wenn er sich selbst direkt oder indirekt aufruft.

Beispiel:

```
void PtrList::Reverse(POSITION p) {
    if (p != PSEUDO) {
        Reverse(Next(p)); // *
        p->Show();         // Atom anzeigen
    }
}
```

Verkettete Liste:



4. Aufruf	Rücksprungadresse *	}	Pop
	NULL		
3. Aufruf	Rücksprungadresse *	}	Pop
	Adresse 3. Zelle: ,T'		
2. Aufruf	Rücksprungadresse *	}	Pop
	Adresse 2. Zelle: ,O'		
1. Aufruf	Rücksprungadresse *		
	Adresse 1. Zelle: ,R'		

STACK

Ausgabe: ,T', ,O', ,R'

Nachteil:

- Stack wird belastet und kann überlaufen.
- Programmausführung im Allgemeinen langsamer als bei iterativen Algorithmen.

Vorteil:

- Einfache Formulierung
- Zwischenspeichern von Daten entfällt.

Wichtig: Jeder rekursive Algorithmus kann durch einen iterativen Algorithmus ersetzt werden.

3.2 Laufzeit rekursiver Algorithmen

Am Beispiel von Reverse()

Sei n Anzahl Listenelemente.

Gesucht: Funktion $T(n)$, welche die Laufzeit von Reverse() „nach oben“ abschätzt.

$n = 0$: $T(0) = A$ konstant (1)

Für $n > 0$ gilt:

$$\begin{aligned}
 T(n) &= B + \underbrace{T(n-1)}_{\text{Reverse(Next(p))}} = \\
 &= B + (B + T(n-2)) = 2 * B + T(n-2) \\
 &\quad \dots \\
 n * b + T(0) &= n * B + A \in O(n)
 \end{aligned}$$

3.3 Rekursives Sortieren von Listen

3.3.1 Sortieren durch Mischen

Verfahren: Falls die Liste mehr als ein Element enthält, teile sie in zwei Hälften. Sortiere die beiden Hälften und mische sie.

Frage: Wie mischt man zwei sortierte Hälften?

Beispiel:

Linke Hälfte	Rechte Hälfte	Ergebnis
<u>31</u> 42 55	<u>27</u> 58 67	27
<u>31</u> 42 55	<u>58</u> 67	27 31
42 55	58 67	

usw.

Laufzeit: $O(n)$

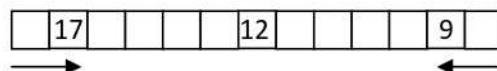
Implementierung:

```
void VekList::MergeSort(POSITION low, POSITION high) {
    if (low < high) {
        POSITION d = (low+high+1) / 2;
        MergeSort(low, d-1);    // Sortiere links
        MergeSort(d, high);     // Sortiere rechts
        Merge(low, high, d);    // Mischen
    }
}
```

Laufzeit: $O(n * \log_2(n))$
 $n = 2^{15}$
 $n^2 = 1.073$ Milliarden Operationen z.B. „Insertion-Sort“
 $n * \log_2(n) = \frac{1}{2}$ Millionen Operationen bei „Merge-Sort“

3.3.2 Sortieren durch Zerlegen

Verfahren: Wähle das mittlere Element in der Liste.
 Durchsuche die Liste von Anfang bis ein größeres Element gefunden und von Ende, bis ein kleineres Element gefunden.
 Tausche beide.
 Suche und tausche weiter, bis sich die Laufindizes treffen.
 Führe den selben Vorgang für den linken und den rechten Teil durch usw.
 Fertig, sobald jeder Teil einelementig ist.



Implementierung:

```
void VekList::QuickSort(POSITION low, POSITION high) {
    ATOM mid = vekPtr[(low+high+1) / 2];    // Mittleres Element
    POSITION i = low;
    POSITION j = high;

    while (i < j) {
        while (vekPtr[i] < mid)
            i++;

        while (vekPtr[j] > mid)
            j--;

        if (i <= j)
            swap(i++, j--);                // Atome tauschen
    }

    if (j > low)
        QuickSort(low, j);

    if (i < high)
        QuickSort(i, high);
}
```

Laufzeit: $O(n^2)$ „worst case“
 $O(n * \log_2(n))$ „average case“

4. Bäume

Wir erhalten eine hierarchische Struktur, wenn wir zulassen dass ein Element mehr als einen Nachfolger hat.

Ein Baum ist eine hierarchische Struktur, wenn außerdem jedes Element höchstens einen Vorgänger hat.

4.1 Der ADT Tree

Definition:

Der ADT TREE besteht aus

- ATOM, POSITION

- (P, R, v) mit

i) P ist eine endliche Teilmenge von POSITION

ii) R ist eine Relation auf P , für die gilt:

1. Es gibt genau eine POSITION $r \in P$, die keinen Vorgänger hat:

Die **Wurzel**

2. Jede Position außer der Wurzel hat genau einen Vorgänger:

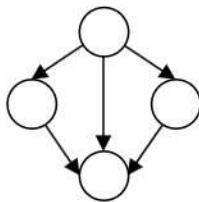
Elternknoten

3. Jede Position außer der Wurzel hat die Wurzel als Vorgänger.

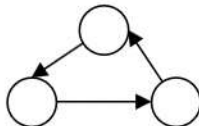
- v ist die Auswertungsfunktion.

Graphik:

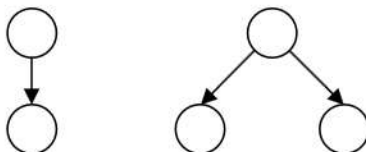
Kein Baum:
Mehrere Vorgänger



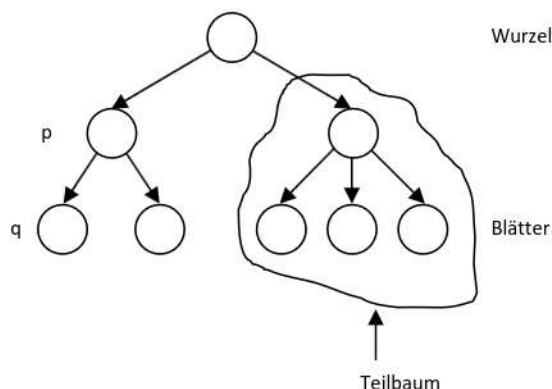
Kein Baum:
Wurzel fehlt



Kein Baum:
Wald!



Baum:
 $h = 2$



Wurzel

Ebene 0

Ebene 1

Ebene 2

Blätter

Teilbaum

Bezeichnungen: vgl. Kopie.

Bezeichnungen im Zusammenhang mit dem abstrakten Datentypen TREE (Kopie):

- Die Positionen in einem Element des abstrakten Datentyps TREE heißen **Knoten**.

Im Folgenden seien p und q Knoten in einem Baum B .

- Der Knoten q heißt **Kindknoten** von p , wenn p der Elternknoten von q ist.
- p und q heißen **Geschwisterknoten**, wenn sie denselben Elternknoten haben.
- Ein Knoten heißt **Blatt**, wenn er keinen Kindknoten besitzt.
- Ein Knoten, der kein Blatt ist, heißt **innerer Knoten**.
- Eine **Kante** verbindet zwei Knoten, von denen der eine Elternknoten des anderen ist.
- Ein **Pfad** ist eine Folge von Kanten, die zwei Knoten verbindet, wobei der eine Knoten Vorgänger des anderen ist.
- Die **Länge eines Pfades** ist die Anzahl der Kanten im Pfad.
- Die **Höhe des Baumes** ist die Länge des längsten Pfades von der Wurzel zu einem Blatt.
- Ein **Teilbaum** von B , der in p wurzelt, besteht aus dem Knoten p und allen Nachfolgern von p .

4.2 k-näre und geordnete Bäume

Definition: Ein k -närer Baum ist ein Baum, in dem jeder Knoten höchstens k Kinder besitzt.

Frage: Wie viele Knoten hat ein k -närer Baum der Höhe h ?

Antwort:

	Min	Max
Ebene 0	1	1
Ebene 1	1	k
Ebene 2	1	k^2
...	...	...
Ebene h	$\frac{1}{h+1}$	$\frac{k^2}{1+k+k^2+\dots+k^h} = \frac{k^{h+1}-1}{k-1} < k^{h+1}$

Folgerungen:

1. Für die Anzahl n der Knoten in einem k -nären Baum der Höhe h gilt:

$$h+1 \leq n < k^{h+1}$$

z.B. $h=5, k=4 \Rightarrow 6 \leq n < 4^6 = 4096$

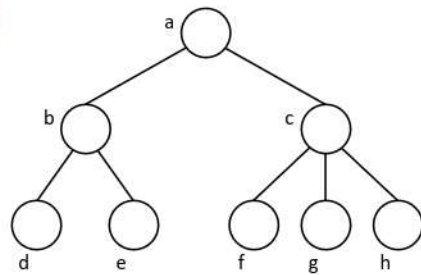
$$\log_k(n) < h+1 \leq n$$

Baum degeneriert zu Liste

Baum mit voller Anzahl an Kindknoten

Definition: Ein k-närer Baum heißt geordnet, wenn in jeder Ebene eine lineare Ordnung Zwischen den Geschwisterknoten existiert.

Beispiel:



d ist linkes Kind, e ist rechtes Kind von b

4.3 Binäre Bäume

Definition: Binärer Baum := geordneter 2-närer Baum + elementare Operationen (vgl. Kopie).

Elementare Operationen für den ADT BINARY TREE (Kopie):

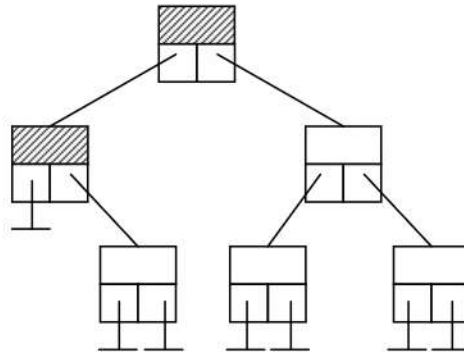
1. Create
Erzeugt einen leeren binären Baum.
2. Root
Bestimmt die Position der Wurzel von T.
3. Parent
Liefert zu einem vorgegebenen Knoten die Position des Elternknotens.
4. LeftChild
Bestimmt zu einem vorgegebenen Knoten im binären Baum die Position des linken Kindknoten.
5. RightChild
Bestimmt zu einem vorgegebenen Knoten im binären Baum die Position des rechten Kindknoten.
6. DeleteSub
Entfernt zu einem vorgegebenen Knoten p im binären Baum den Teilbaum, dessen Wurzel p ist, sowie den Knoten p selbst.
7. ReplaceSub
Fügt einen binären Baum S beim Knoten p im binären Baum ein, wobei die Wurzel von S den Knoten p ersetzt. Diese Routine wird normalerweise verwendet, wenn p ein Blatt ist oder wenn der binäre Baum leer ist.
8. Merge
Hängt einen binären Baum L als linken Teilbaum und einen binären Baum R als rechten Teilbaum an die Wurzel des binären Baumes.
9. Retrieve
Liefert zu einem vorgegebenen Knoten im binären Baum das entsprechende Atom.
10. Update
Ersetzt das Atom in einem vorgegebenen Knoten durch ein anderes Atom.

4.3.1 Implementierung

a) Dynamisch

Es wird eine verkettete Folge von Knoten gebildet, wobei jeder Knoten einen Zeiger auf das rechte Kind und einen Zeiger auf das linke Kind besitzt.

Graphik:



Klassendefinition: vgl. Kopie (nächste Seite!)

Dynamische Implement eines binären Baumes (Kopie):

```

// Header-Datei node.h
// Enthält die Klassen Cell und Node zur Darstellung der Knoten in einem
// binären Baum, der dynamisch implementiert wird.

#include <cstdio>          // Def. symb. Konstante NULL.

using namespace std;

#define PSEUDO NULL        // Pseudo-Position: NULL-Zeiger.

typedef int ATOM;          // Datentyp für Infos.

class Cell {
private:
    Cell *leftsuc, *rightsuc;
public:
    Cell(Cell *lp = PSEUDO, Cell *rp = PSEUDO) {
        leftsuc = lp;
        rightsuc = rp;
    }

    virtual ~Cell() { }

    void SetLeftSuc(Cell *p) { leftsuc = p; }
    void SetRightSuc(Cell *p) { rightsuc = p; }
    Cell** GetLeftSuc(void) const { return &leftsuc; }
    Cell** GetRightSuc(void) const { return &rightsuc; }
};

class Node : public Cell {
private:
    ATOM key;
public:
    Node(ATOM a, Cell *lp = PSEUDO, Cell *rp = PSEUDO) : Cell(lp, rp) {
        key = a;
    }

    ATOM GetKey(void) const { return key; }
    void SetKey(ATOM neu) { key = neu; }
};

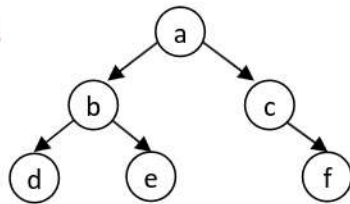
typedef Cell* POSITION;

```

b) Statisch

Die statische Darstellung wird mit Hilfe eines Vektors imitiert. Ein Vektorelement speichert neben dem Atom zwei Indizes vom rechten bzw. linken Kindknoten.

Graphik:



		li	re
0	a	1	2
1	b	3	4
2	c	-1	5
3	d	-1	-1
4	e	-1	-1
5	f	-1	-1

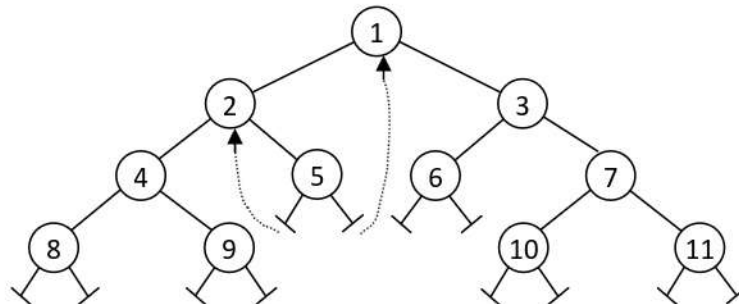
4.3.2 Durchlaufstrategien

a) Preorder – Schema

Methode: Beginnend bei der Wurzel wird
 - zuerst der Knoten selbst
 - dann der linke [rechte] Teilbaum des Knotens
 - dann der rechte [linke] Teilbaum des Knotens inspiziert.

Bemerkung: Das Preorder – Schema heißt auch NLR – Ordnung oder NRL – Ordnung.

Beispiel:



NLR – Durchlauf: 1, 2, 4, 8, 9, 5, 3, 6, 7, 10, 11

Implementierung:

```

void BINARY_TREE::NLR_order(POSITION p) {
    if (p != PSEUDO) {
        Inspect(p);
        NLR_order(LeftChild(p));
        NLR_order(RightChild(p));
    }
}
  
```

// tu, was möglich ist
 // linker Teilbaum
 // rechter Teilbaum

Anwendung:

1. Speichern von Atomen aus binären Bäumen in Dateien.
2. Suche in binären Bäumen.

b) Inorder – Schema

Methode: Beginnend bei der Wurzel wird
- zuerst der linke [rechte] Teilbaum
- dann der Knoten selbst
- dann der rechte [linke] Teilbaum des Knotens inspiziert.

Bemerkung: Das Inorder – Schema heißt auch LNR – Ordnung oder RNL – Ordnung.

LNR – Durchlauf: 8, 4, 9, 2, 5, 1, 6, 3, 10, 7, 11

Anwendungen:

1. Sortieren von Atomen in binären Suchbäumen.
2. Fädelung binärer Bäume.

c) Postorder – Schema

Methode: Beginnend bei der Wurzel wird
- zuerst der linke [rechte] Teilbaum
- dann der rechte [linke] Teilbaum
- dann der Knoten selbst inspiziert.

Bemerkung: Das Postorder – Schema heißt auch LRN – Ordnung oder RLN – Ordnung.

LRN – Durchlauf: 8, 9, 4, 5, 2, 6, 10, 11, 7, 3, 1

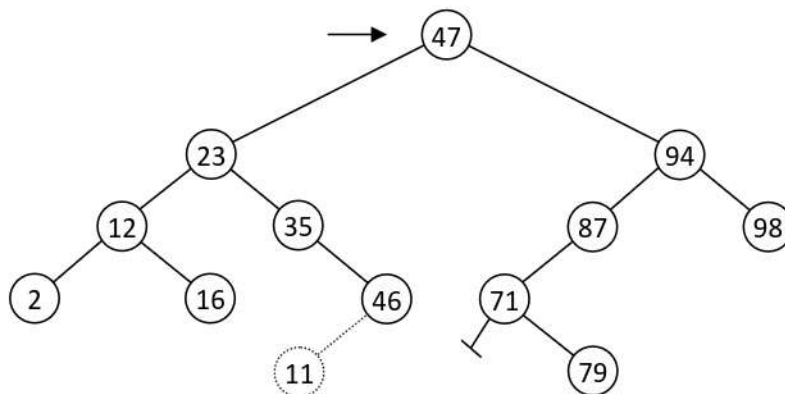
Anwendungen:

Löschen von Teilbäumen.

4.3.3 Binäre Suchbäume

Definition: Ein binärer Suchbaum ist ein binärer Baum mit folgender Eigenschaft:
Der Wert des Schlüssels in einem Knoten ist größer als die Werte aller Schlüssel im Linken Teilbaum und kleiner als alle Schlüssel im rechten Teilbaum.

Beispiel:



LNR - Durchlauf: 2, 12, 16, 23, 35, 46, 47

Bemerkung: LNR – Durchlauf liefert aufsteigende Reihenfolge der Atome:
„Tree-Sort“ – Algorithmus

Laufzeit: $O(n * \log_2(n))$ „normalerweise“

Klassendefinition: vgl. Kopie

Dynamische Implementierung eines binären Suchbaums (Kopie):

```
// Header-Datei searchtr.h
// Enthält die Klasse SearchTree zur Darstellung eines binären
// Suchbaumes, der dynamisch implementiert wird.

#include "node.h"          // Def. der Klassen Cell, Node.

class SearchTree {
private:
    POSITION start;  // Zeiger auf Wurzel.

    // Hilfsmethoden:
    POSITION SearchRek(ATOM k, POSITION p);
    void InsRek(ATOM neu, POSITION *p);
    void DelRek(POSITION *p);          // Löscht Teilbaum
    void DelRek(ATOM k, POSITION *p);  // Löscht Knoten
    void CopyRek(POSITION *p);

    ATOM DeleteMin(POSITION *p);

public:
    SearchTree() { start = PSEUDO; }
    SearchTree(const SearchTree& st); // Kopierkonstruktor
    ~SearchTree();

    POSITION GetRoot(void) { return start; }

    // Überladene Zuweisung
    SearchTree& operator=(const SearchTree& st);

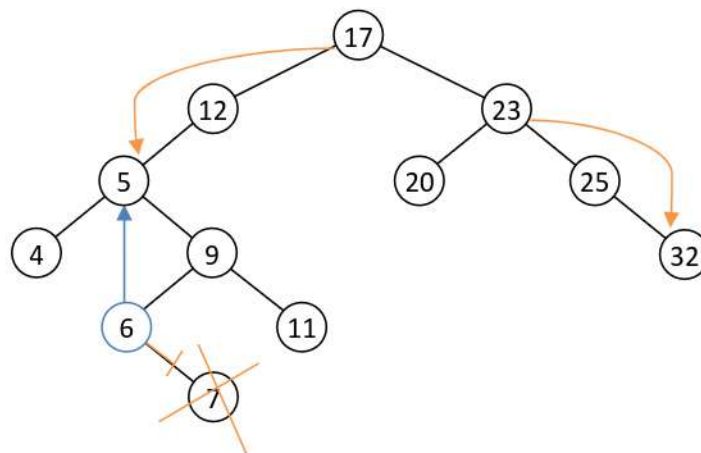
    // Elementare Methoden:
    POSITION Find(ATOM k);
    void Update(ATOM k, POSITION p);
    void Insert(ATOM neu);
    void Delete(ATOM k);
    ATOM Retrieve(POSITION ptr);
};
```

Einfügen:Pseudo – Code:

Falls Teilbaum leer: Knoten mit neuem Atom anlegen und im Elternknoten einklinken.
 Andernfalls: Schlüssel < Schlüssel im aktuellen Knoten?
 Ja: Im linken Teilbaum einfügen.
 Nein: Im rechten Teilbaum einfügen.

Implementierung:

```
void SearchTree::InsRek(ATOM a, POSITION *p) {
    if (*p == PSEUDO) {           // Teilbaum leer
        *p = new Node(a);
    } else {
        if (a < ((Node*)*p)->GetKey()) {
            InsRek(a, (*p)->GetLeftSuc());
        } else {
            if (a > ((Node*)*p)->GetKey()) {
                InsRek(a, (*p)->GetRightSuc());
            } else {
                // keine doppelten Schlüssel
            }
        }
    }
}
```

Löschen:

1. Fall: „keine Kinder“, z.B. 7
 Zeiger im Vorgänger auf NULL und Speicherplatz freigeben.
2. Fall: „nur ein Kind“ z.B. 25 oder 12
 Anhängenden Teilbaum im Elternknoten einklinken. Knoten löschen.
3. Fall: „2 Kinder“ z.B. 5
 Suche im rechten Teilbaum kleinsten Schlüssel:
 Dieser hat höchstens ein Kind!
 Kopiere den dortigen Schlüssel in den zu Löschenden Knoten und lösche den Knoten des kleinsten Schlüssels.

Laufzeit:

von Einfügen, Löschen, Hervorheben:
 $O(d)$, d Höhe des Baumes.

Was lässt sich über d sagen?

Wir wissen: $\underbrace{\log_2(n) < d}_{\text{In diesem Fall degeneriert zu Liste}} + 1 \leq n$

„normalerweise“: $d = k * \log_2(n)$ $k \in \mathbb{N}$ klein

$\Rightarrow$ Laufzeit: $O(\log_2(n))$ normalerweise

4.3.4 Warteschlangen und Prioritäten

Definition: Der ADT PriorityQueue besteht aus einer linear geordneten Menge von Atomen und folgenden Operationen:

1. Create - (wie gehabt).
2. Insert - Fügt ein Atom ein.
3. DeleteMin - Löscht das „kleinste“ Atom.

Beispiel: Notaufnahme im Krankenhaus.

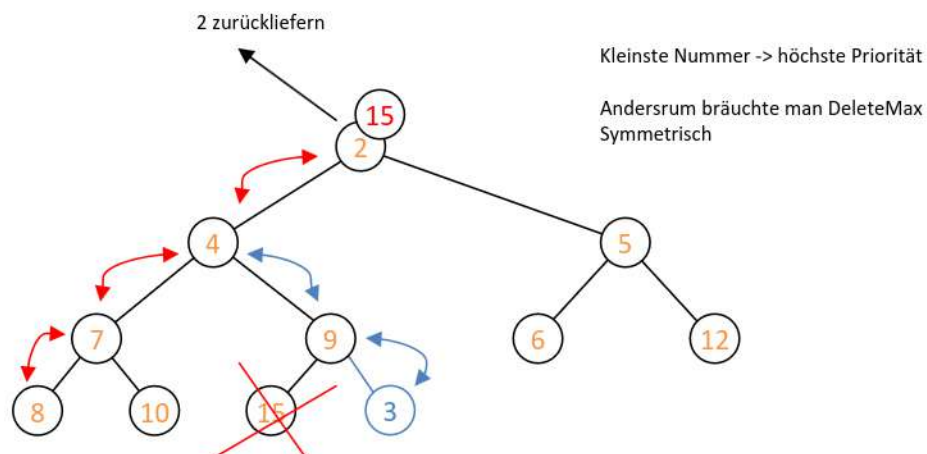
Frage: Welche Datenstruktur ist zur Darstellung einer PriorityQueue geeignet?

Möglich $\begin{cases} \text{Sortierte Liste} & \text{(lineare Laufzeit beim suchen)} \\ \text{Binärer Suchbaum} & \text{(Risiko: kann zu Liste degenerieren)} \end{cases}$

Wir machen es anders: Heap

- Definition:**
1. Ein binärer Baum heißt optimal, wenn jeder innere Knoten – außer evtl. in der vorletzten Ebene – zwei Kindknoten hat.
 2. Ein binärer Baum heißt linksorientiert, wenn alle Knoten in einer bestimmten Ebene so weit wie möglich links liegen.

Beispiel:



Definition: Ein linksorientierter, optimaler binärer Baum erfüllt die Heap-Bedingung:
 $\Leftrightarrow$ Der Schlüssel in einem Knoten ist kleiner als in allen Kindknoten.

Bemerkung: Ein linksorientierter, optimaler binärer Baum, der die Heap-Bedingung erfüllt, heißt Heap.

Einfügen in Heap (Insert):

- Füge neuen Knoten mit Schlüssel in der untersten Ebene am weitesten links ein.
 - „Bubble-Up“ des Schlüssels mit Schlüssel im Elternknoten bis HeapBedingung erfüllt.
- Laufzeit: $O(h)$

Löschen im Heap (DeleteMin):

- Kopiere den Schlüssel im „letzten“ Knoten - d.i. der Knoten in der untersten Reihe am weitesten rechts – in die Wurzel.
- „Bubble-Down“ mit dem kleineren Schlüssel in den Kindknoten bis Heap-Bedingung erfüllt.

Implementierung:

Mit dem Ziel, den Heap als Array zu speichern, nummerieren wir die Knoten im Baum durch:

- Die Wurzel erhält die Nummer 0.
- Das linke Kind der Wurzel die Nummer 1, das rechte Kind die Nummer 2.
- In jeder nachfolgenden Ebene werden die Knoten von links nach rechts durchnummeriert.

Wichtig:

Mit dieser linearen Ordnung gilt:

1. Für den Knoten mit der Nummer i ist $2i + 1$, bzw. $2i + 2$ die Nummer des linken bzw. rechten Kindes.
2. Für den Knoten mit der Nummer $i > 0$ ist $\frac{(i-1)}{2}$ die Nummer des Elternknotens.

Dann können die Knoten in einem Vektor gespeichert werden, wobei jedes Vektorelement lediglich Atome speichert.

(Wegen Löchern in binären Bäumen und weil binäre Bäume nicht linksorientiert und optimal sind, geht dies bei binären Bäumen nicht!!!)

Klassendefinition: vgl. Kopie**Kopie:**

```
// Header-Datei priqueue.h
// Enthält das Klassen-Template PrioQueue zur Darstellung einer
// PriorityQueue, die vektoriell implementiert wird.

template<class T>
class PrioQueue {
private:
    int size;    // Anzahl Elemente in der Queue.
    int max;     // Maximale Anzahl Vektorelemente.
    T *vekPtr;   // Zeiger auf Vektor.

    // Zum Bubbeln:
    void swap(T &x, T&y) {T z = x; x = y; y = z; }

public:
    PrioQueue(int n = 255); // Konstruktor: erzeugt Vektor mit n
                           // Elementen,
                           // Defaultmäßig wird Speicherplatz für 255
                           // Elemente angelegt.

    // Kopierkonstruktor.
    PrioQueue(const PrioQueue<T>&);
    // Zuweisung.
    const PrioQueue<T>& operator=(const PrioQueue<T>&);
    // Destruktor.
    ~PrioQueue() { delete[] vekPtr; }

    int GetMax() const { return max; }
    int GetSize() const { return size; }
    void SetSize(int n) { size = n; }
    T* Head() const { return vekPtr; }
    void Insert(const T& a);
    T* DeleteMin();
    void HeapSort();
    void PutVek();
};
// Explizite Instatiation für den Typ int
template <class T> class PrioQueue<int>;
```

Laufzeit: Einfügen und Löschen in Heaps:
 $O(d)$, d Höhe des Heaps

Behauptung: $d \leq \log_2(n)$
 n Anzahl Knoten im Heap

Beweis:

	Anzahl Knoten mindestens
Ebene 0	1
Ebene 1	2
Ebene 2	2^2
...	
Ebene $d - 1$	2^{d-1}
Ebene d	1 (min)

$$1 + 2^1 + 2^2 + \dots + 2^{d-1} + 1 = \frac{2^d - 1}{2 - 1} + 1 = 2^d$$

$\Rightarrow n \geq 2^d \Rightarrow \log_2(n) \geq d$ ✓
 $\Rightarrow$ Laufzeit Einfügen und Löschen $O(\log_2(n))$

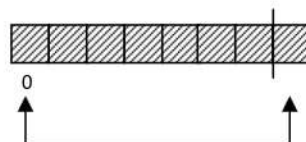
Anwendung: „Heap-Sort“ Algorithmus.

Verfahren: **1. Schritt:** Mithilfe von Insert(), werden alle zu sortierenden Atome in einen Heap eingefügt.

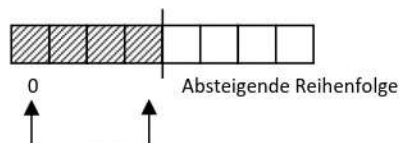
2. Schritt: Mit DeleteMin() werden die Atome dem Heap in aufsteigender Reihenfolge entnommen.

Hierbei wird kein zweites Array zum Aufnehmen der bereits sortierten Elemente benötigt:

Heap nach dem 1. Schritt:



Später:



Laufzeit von „Heap-Sort“:

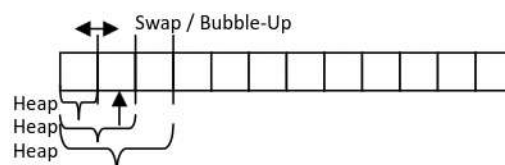
Gibt es n zu sortierende Atome

$\Rightarrow$ n Aufrufe von Insert() á $O(\log_2(n))$
 n Aufrufe von DeleteMin() á $O(\log_2(n))$

$\Rightarrow$ Laufzeit $\in O(n * \log_2(n))$

Immer und es kann im selben Array sortiert werden!!!

Unsortiertes Array:



Bis auf ein paar Ausnahmesituationen ist der QuickSort schneller, HeapSort ist aber stabiler

4.4 B – Bäume und B'-Bäume

Vorbemerkung:

1. B-Bäume wurden 1972 von R. Bayer und E.N. McGeight entworfen.
2. Sie sind die am meisten anerkannte Datei-Indexstruktur.

Vorteile:

- Schneller Zugriff.
- Reorganisation im Allgemeinen nur lokal nötig.
- Gute Speicherausnutzung.

Definition: B-Baum vgl. Kopie.

Definition eines B-Baumes (Kopie):

Ein **B-Baum** (engl. B-Tree) **der Ordnung d** ist ein $(2d + 1)$ -närer Baum, mit folgenden Eigenschaften:

1. Die Schlüssel in jedem Knoten besitzen eine lineare Ordnung.
2. Jeder innere Knoten besitzt ein Kind mehr als Datenelemente.
3. Die Wurzel besitzt zwischen 1 und $2d$ Datenelemente.
4. Jeder Knoten außer der Wurzel besitzt zwischen d und $2d$ Datenelemente.
5. Alle Blätter liegen in derselben Ebene des Baumes.
6. Es gilt die verallgemeinerte Suchbaumeigenschaft:
Sind die Schlüssel $K_1, \dots, K_m$ in einem Knoten K in aufsteigender Reihenfolge sortiert, so gibt es eine lineare Teilordnung unter den Teilbäumen $S_0, \dots, S_m$ von K wie folgt:
 - a) Jeder Schlüssel von S_0 ist kleiner oder gleich K_1 .
 - b) Jeder Schlüssel von S_m ist größer als K_m .
 - c) Jeder Schlüssel in S_i ($0 < i < m$) liegt strikt zwischen K_i und K_{i+1} .

Satz: Für die Höhe h eines B-Baumes der Ordnung $d \geq 2$ mit N Datenelementen gilt:

$$h \leq 1 + \log_{d+1} \left(1 + \frac{(N-1)}{d} \right)$$

(Höhe ist maßgeblich für die Laufzeit)

Was lässt sich über die Höhe eines B-Baumes sagen?

Vgl. Satz (Kopie gleich hier drüber)

„To get a feeling“:

	N	10^6	10^9	10^{12}
d				
16		$h \leq 4$	$h \leq 7$	$h \leq 9$
32		$h \leq 3$	$h \leq 5$	$h \leq 7$
64		$h \leq 3$	$h \leq 4$	$h \leq 6$

Einfügen in B-Bäumen:

Falls Baum leer: Neue Wurzel mit einzufügendem Element anlegen. Fertig.
Andernfalls: Das Blatt und die Position im Blatt suchen, wo das Element einzufügen ist.
Platz im Blatt?:
Ja: Element einfügen. Fertig.
Nein (Überlauf): Knoten Splitten, Median im Elternknoten einfügen.
Überlauf kontrollieren.

Falls Wurzel gesplittet wurde, neue Wurzel mit Median erzeugen.
-> Die Höhe des Baumes wächst um 1!

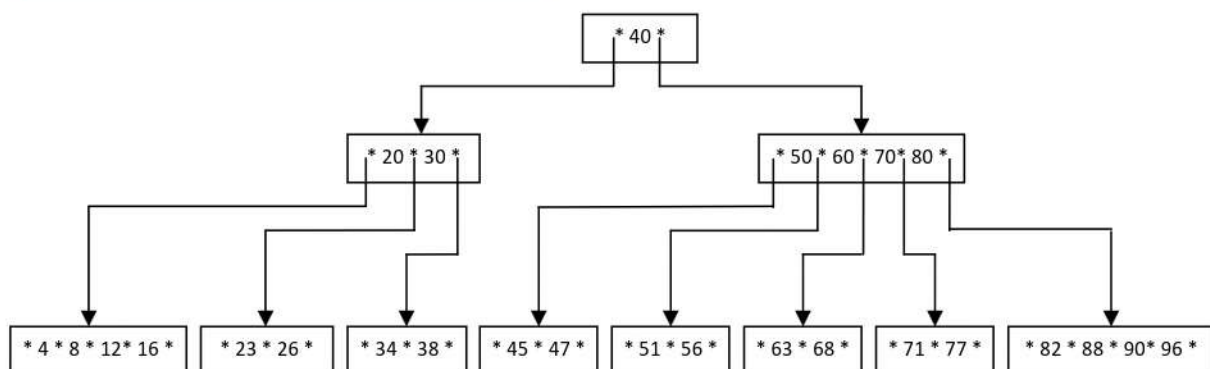
Löschen in B-Bäumen:

Beginnend bei der Wurzel den zu löschenden Knoten suchen.
Falls in einem Blatt gefunden, dort löschen.
Andernfalls: Im rechten Teilbaum nächst größeren Schlüssel suchen. Dieser befindet sich in einem Blatt!
Beide Schlüssel tauschen und Schlüssel im Blatt löschen.

„Unterlauf“ kontrollieren.
Ist Unterlauf aufgetreten: Vom Nachbarn „borgen“ oder Knoten zusammenlegen und mittleres Element aus Elternknoten einfügen und im Elternknoten löschen.
Unterlauf kontrollieren!

Ist Unterlauf in der Wurzel aufgetreten (d.h. Wurzel Leer) so ist zusammen gelegter Kindknoten die Neue Wurzel. Speicher für „alte“ Wurzel freigeben.
-> Die Höhe des Baumes vermindert sich um 1!

Ein B-Baum der Ordnung 2 ($d = 2$) (Kopie):



Laufzeit: Einfügen, Löschen, Suchen:
 $O(h)$, h Höhe des Baumes

Implementierung: vgl. Kopie.

```
// Header-Datei b_tree.h
// Enthält die Klassen Node und B_Tree zur Darstellung
// eines Bayer-Baumes.

#include <stdio.h>                // Def. der symb. Konstanten NULL

#define PSEUDO NULL              // Pseudo-Position: NULL-Zeiger.

// Die Klasse Node stellt einen Knoten im B-Baum dar.
template<class T, int d>
class Node {
private:
    T keydat[2*d];                // Vektor für Datenelemente.
    Node* child[2*d+1];          // Zeiger auf Nachfolger.
    int size;                     // aktuelle Anzahl Knotenelemente.

public:
    Node();                      // Standardkonstruktor:
                                // Setzt alle child-Elemente auf NULL.

    int GetSize() const { return size; }
    void SetSize(int i) { size = i; }

    T GetKey(int i) const { return *(keydat+i); }
    void SetKey(int i, T neu) { keydat[i] = neu; }

    Node* GetChild(int i) const { return *(child+i); }
    void SetChild(int i, Node* p) { child[i] = p; }
};

#define POSITION Node<T, d>*
```

```

// Die Klasse B_Tree stellt einen Bayer-Baum dar.
#include "b_tree.h"

template<class T, int d>
class B_Tree {
private:
    POSITION root;           // Zeiger auf Wurzel.

    // Die von Insert verwendeten Methoden:
    int AddAtom(T*, POSITION, int, POSITION*, T*);
    int InsRek(T*, POSITION, POSITION*, T*);
    int NewRoot(T*, POSITION);

    // Die von Delete verwendeten Methoden:
    int DelRek(T, POSITION);
    int RestoreTree(POSITION p, int);
    void DelRekNode(POSITION);

    // Hilfsmethoden:
    void shiftr(POSITION, int);           // Recht shiften ab Index i.
    void shiftl(POSITION, int);          // Links shiften ab Index i.

public:
    B_Tree() { root = PSEUDO; }           // Standardkonstruktor.
    B_Tree(const B_Tree<T>&);             // Kopierkonstruktor.
    const B_Tree<T>& operator=(const B_Tree<T>&); // Zuweisung.

    ~B_Tree();                           // Löscht alle Knoten im Baum.

    POSITION GetRoot(void) { return root; }

    // Elementare Methoden:
    int Insert(const T& a);
    int Delete(const T& a);
    POSITION Search(const T& a);
};

template class B_Tree<int, 16>;           // Explizite Instantiierung.

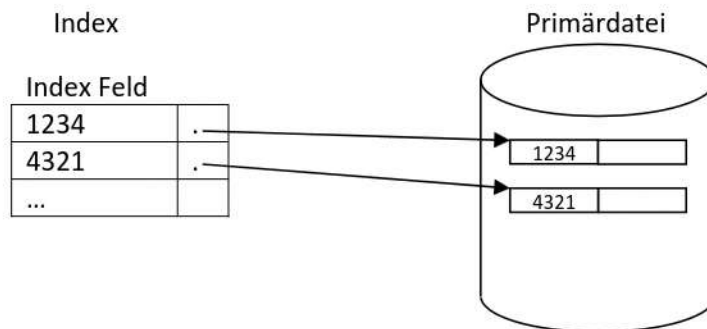
```

Anwendung: Dynamischer Multi-Level Index.

Voraussetzung: Gegeben sei eine Datei, sog. Primärdatei, in der jeder Datensatz über einen Schlüssel eindeutig identifiziert werden kann.

Definition: Ein Index besteht aus einer Menge von Paaren:
 - einem Index-Feld, das einen Schlüssel speichert.
 - der Adresse („Blockzeiger“) des entsprechenden Datensatzes in der Datei.

Graphik:



Vorteil: Schneller Zugriff auf Datensätze

Definition: Dynamischer Multi-Level Index : $\Leftrightarrow$ Index, der als B- oder als B'-Baum implementiert ist.

a) Implementierung als B-Baum

Datenelement : = Index-Feld + Blockzeiger

Frage: Welche Ordnung d ist für den Index geeignet?

Antwort: Wähle d so groß, dass ein Knoten einen Block ausfüllt.

Beispiel:

Blockgröße:	1024 Byte
Index Feld:	8 Byte
Größe Zeiger:	4 Byte
Variable für aktuelle Anzahl:	4 Byte

$$2d * 12 + (2d + 1) * 4 + 4 \leq 1024$$

$$\Rightarrow 32d \leq 1016 \Rightarrow d \leq 31.75$$

$$\Rightarrow d = 31$$

Frage: Es soll ein B-Baum der Ordnung $d = 31$ konstruiert werden. Wie viele Index-Einträge passen in 4 Ebenen, wenn jeder Knoten im Schnitt $\frac{2}{3}$ voll ist.

Antwort: Jeder Knoten enthält im Schnitt $\frac{2}{3} * 2d = 41$ Index-Einträge.

	Anzahl Knoten	Anzahl Index-Einträge	Anzahl Zeiger Kindknoten
Ebene 0	1	41	42
Ebene 1	42	$41 * 42$	42^2
Ebene 2	42^2	$41 * 42^2$	42^3
Ebene 3	42^3	$41 * 42^3$	
		3.111.695	

Frage: Wie viele Blockzugriffe sind erforderlich um einen Datensatz aus der Primärdatei hervorzuholen?

Antwort: $\underbrace{4}_{\text{Index}} + \underbrace{1}_{\text{Primärdatei}} = 5$
 Um an Index ranzukommen (im schlimmsten Fall).
 1 wenn man in der Wurzel schon fündig wird!

b) Implementierung als B'-Baum

Definition: vgl. Kopie.

Definition eines B'-Baumes (Kopie):

Ein B'-Baum (engl. auch B+-Tree) der Ordnung d ist ein $(2d + 1)$ -ärer Baum, bei dem die Struktur eines Blattknotens verschieden von der Struktur eines inneren Knotens ist. Und zwar gilt:

- a) Für die **inneren** Knoten:
- o Jedes Datenelement enthält nur ein Index-Feld.
 - o Die Index-Felder sind linear geordnet.
 - o Jeder Knoten besitzt ein Kind mehr als Datenelemente.
 - o Jeder Knoten außer der Wurzel besitzt zwischen d und $2d$ Datenelemente.
 - o Die Wurzel besitzt zwischen 1 und $2d$ Datenelemente.
 - o Es gilt die (modifizierte) verallgemeinerte Suchbaumeigenschaft:
 Sind die Index-Felder $K_1, \dots, K_m$ in einem inneren Knoten K in aufsteigender Reihenfolge sortiert, so gibt es eine lineare Teilordnung unter den Teilbäumen $S_0, \dots, S_m$ von K wie folgt:
 - i) Jedes Index-Feld von S_0 ist kleiner oder gleich K_1 .
 - ii) Jedes Index-Feld von S_m ist größer als K_m .
 - iii) Für jedes Index-Feld F von S ($0 < i < m$) gilt: $K_i < F \leq K_{i+1}$.
- b) Für die **Blätter**:
- o Jedes Datenelement enthält neben dem Index-Feld einen Blockzeiger, der den entsprechenden Datensatz in der Primärdatei adressiert (oder einen Block von Blockzeigern auf Datensätze in der Primärdatei mit gleichem Schlüssel).
 - o Zu jedem Index-Feld gibt es ein Datenelement in einem Blatt.
 - o Die Index-Felder sind linear geordnet.
 - o Jedes Blatt besitzt zwischen d und $2d$ Datenelemente.
 - o Die Blätter sind – unter Beibehaltung der linearen Ordnung der Index-Felder – miteinander verkettet.
 - o Jedes Blatt liegt in derselben Ebene des Baumes.

Darstellung der Knoten:

Wie bei B-Baum + Flag zur Unterscheidung interner / externer Knoten.

1. **Frage:** Wie groß ist die Ordnung d für einen B'-Baum zu wählen?

Antwort: Ein Knoten sollte einen Block füllen.

Beispiel:

Blockgröße:	1024 Byte
Index Feld:	8 Byte
Größe Zeiger:	4 Byte
Variable für aktuelle Anzahl:	4 Byte
Länge Flag:	1 Byte

$$\begin{aligned}2d * 8 + (2d + 1) * 4 + 4 + 1 &\leq 1024 \\ \Rightarrow d &\leq 1015/24.0 \\ \Rightarrow d &= 42\end{aligned}$$

2. **Frage:** Wie viele Index-Einträge können in einem B'-Baum der Höhe $h = 3$ untergebracht werden, wenn jeder Knoten im Schnitt $\frac{2}{3}$ voll ist?

Antwort: Jeder Knoten enthält im Schnitt $\frac{2}{3} * 2d = 56$ Schlüssel.

	Anzahl Knoten	Anzahl Schlüssel	Anzahl Zeiger Kindknoten
Ebene 0	1	56	57
Ebene 1	57	$56 * 57$	57^2
Ebene 2	57^2	$56 * 57^2$	57^3
Ebene 3	57^3	$56 * 57^3$	
		10.370.808	

Hinweis: Ein B*-Baum ist ein spezieller B-Baum, wobei gilt:
Jeder Knoten außer der Wurzel ist mindestens $\frac{2}{3}$ voll.